

Dr. Isah Charles Saidu

Lead Instructor

Liatris Systems

AI & Technology Solutions

Session Duration: 1 Hour 30 minutes

July 5, 2026



Liatris Systems



In partnership with the Independent National Electoral Commission

Session Roadmap

Part 1: NLP Fundamentals

- ⚙️ What is NLP?
- 🏗️ NLP Pipeline
- 📄 Text Preprocessing

Part 2: Traditional NLP

- ★ Bag of Words
- TF-IDF
- 🎮 Word Embeddings

Part 3: Transformers

- ⚙️ Attention Mechanism
- 🏗️ Transformer Architecture
- ★ BERT, GPT, LLMs

Part 4: Modern Applications

- 💬 Chatbots & RAG
- ✅ Sentiment Analysis
- 🛡️ Misinformation Detection

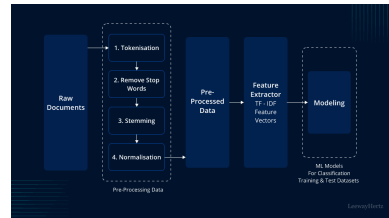
What is NLP?

What is Natural Language Processing?

Natural Language Processing (NLP) is a field of AI that enables computers to **understand, interpret, and generate** human language.

NLP Applications

- **Text Classification** – Categorize documents
- **Named Entity Recognition** – Extract entities
- **Sentiment Analysis** – Detect opinions
- **Machine Translation** – Translate languages
- **Question Answering** – Answer questions
- **Text Generation** – Generate content



Why NLP is Challenging

Language Complexity

- **Ambiguity** – Multiple meanings
- **Context** – Meaning depends on context
- **Sarcasm** – Opposite meaning
- **Idioms** – Non-literal meaning
- **Evolution** – Language changes over time

💡 Example: Ambiguity

- "Time flies like an arrow"
- "Fruit flies like a banana"

NLP Goals

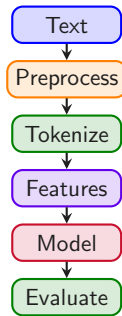
- Understand context
- Handle ambiguity
- Process multiple languages
- Learn from examples

NLP Pipeline

NLP Pipeline

Pipeline Steps

1. **Text Acquisition** – Collect text data
2. **Preprocessing** – Clean and normalize
3. **Tokenization** – Split into tokens
4. **Feature Extraction** – Convert to vectors
5. **Modeling** – Apply ML/DL models
6. **Evaluation** – Assess performance



Text Preprocessing

Text Preprocessing Techniques

Lowercasing

- Convert all text to lowercase
- **Before:** "NLP is Amazing!"
- **After:** "nlp is amazing!"

Stopword Removal

- Remove common words
- **Stopwords:** "the", "is", "and"
- **Before:** "The cat is sleeping"
- **After:** "cat sleeping"

Stemming & Lemmatization

- Reduce to root form
- **Stemming:** "running" → "run"
- **Lemmatization:** "better" → "good"

Tokenization

What is Tokenization?

- Splitting text into tokens
- Tokens = words, subwords, or characters
- Essential first step in NLP

</> Code Example

```
1 from nltk.tokenize import word_tokenize,
   sent_tokenize
2 from transformers import AutoTokenizer
3
4 # Word tokenization
5 text = "NLP is transforming elections!"
6 tokens = word_tokenize(text)
7 print(tokens)
8 # Output: ['NLP', 'is', 'transforming', 'elections', '!']
9
10 # Sentence tokenization
11 sentences = sent_tokenize("Hello. How are you?")
12 print(sentences)
13 # Output: ['Hello.', 'How are you?']
14
15 # Transformer tokenizer
16 tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
17 tokens = tokenizer.tokenize("NLP is amazing!")
18 print(tokens)
19 # Output: ['nlp', 'is', 'amazing', '!']
```

Traditional NLP

Bag of Words (BoW)

What is BoW?

- Represent text as word frequency
- Ignore grammar and word order
- Create vocabulary of unique words
- Each document = vector of word counts

</> Code Example

```
1 from sklearn.feature_extraction.text import
   CountVectorizer
2
3 docs = [
4     "NLP is amazing",
5     "Machine learning is great",
6     "NLP and ML are powerful"
7 ]
8
9 vectorizer = CountVectorizer()
10 X = vectorizer.fit_transform(docs)
11
12 # Get feature names
13 features = vectorizer.get_feature_names_out()
14 print(features)
15 # Output: ['amazing', 'are', 'great', ...]
16
17 # Document vectors
18 print(X.toarray())
19 # [[1 0 0 0 1 0 1 0]
20 #  [0 0 1 0 0 0 0 1]
21 #  [0 1 0 1 0 1 0 0]]
```

TF-IDF (Term Frequency-Inverse Document Frequency)

What is TF-IDF?

- Measures word importance
- **TF** – Word frequency in document
- **IDF** – Inverse document frequency
- Emphasizes important words

$$TF\text{-}IDF = TF \times IDF$$

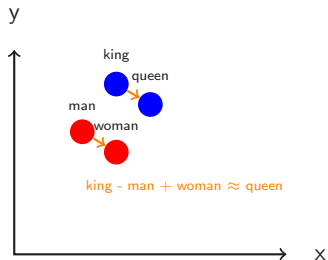
</> Code Example

```
1 from sklearn.feature_extraction.text import
   TfidfVectorizer
2
3 docs = [
4     "NLP is amazing",
5     "Machine learning is great",
6     "NLP and ML are powerful"
7 ]
8
9 vectorizer = TfidfVectorizer()
10 X = vectorizer.fit_transform(docs)
11
12 # Get feature names
13 features = vectorizer.get_feature_names_out()
14 print(features)
15
16 # TF-IDF values
17 print(X.toarray())
18 # [[0.5 0.  0.  0.  0.5 0.  0.5 0. ]
19 #   [0.  0.  0.5 0.  0.  0.  0.  0.5]
20 #   [0.  0.5 0.  0.5 0.  0.5 0.  0. ]]
```

Word Embeddings

What are Word Embeddings?

- Dense vector representations
- Capture semantic meaning
- Similar words have similar vectors
- **Examples:**
 - Word2Vec
 - GloVe
 - FastText



Transformers

The Transformer Revolution

Key Milestones

- **2017** – "Attention is All You Need"
- **2018** – BERT introduced
- **2019** – GPT-2 released
- **2020** – GPT-3 & T5
- **2022** – ChatGPT revolution
- **2023** – GPT-4, Llama, Claude

Why Transformers?

- **Parallelization** – Faster training
- **Context** – Better understanding
- **Transfer Learning** – Pre-training
- **Scalability** – Billions of parameters
- **State-of-the-art** – All NLP tasks

Attention Mechanism

What is Attention?

- Focus on relevant parts
- Weigh importance of words
- Context-aware representation
- **Key Idea:**
 - Query, Key, Value
 - Softmax for weights

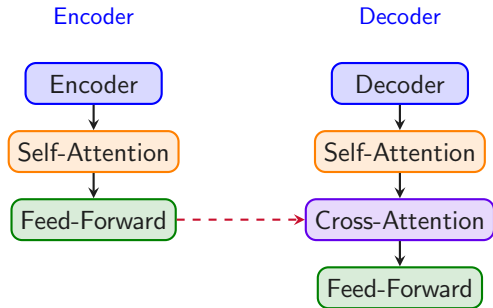


Attention focuses on "cat sat"

Transformer Architecture

Architecture Components

- **Encoder** – Processes input
- **Decoder** – Generates output
- **Multi-Head Attention** – Multiple attention heads
- **Feed-Forward** – Position-wise processing
- **Positional Encoding** – Word order



BERT, GPT, and LLMs

BERT (Bidirectional Encoder Representations from Transformers)

What is BERT?

- Bidirectional understanding
- Masked Language Modeling (MLM)
- Next Sentence Prediction (NSP)
- Pre-trained on Wikipedia & Books
- **Key Feature:** Contextual embeddings

</> BERT Example

```
1 from transformers import BertTokenizer, BertModel
2
3 tokenizer = BertTokenizer.from_pretrained(
4     'bert-base-uncased'
5 )
6 model = BertModel.from_pretrained(
7     'bert-base-uncased'
8 )
9
10 text = "NLP is transforming elections"
11 tokens = tokenizer(
12     text,
13     return_tensors='pt',
14     padding=True,
15     truncation=True
16 )
17 # Get contextual embeddings
18 outputs = model(**tokens)
19 embeddings = outputs.last_hidden_state
20 print(f"Embedding shape: {embeddings.shape}")
21 # Output: (1, 6, 768)
```

GPT (Generative Pre-trained Transformer)

⚙️ What is GPT?

- Autoregressive language model
- Unidirectional (left-to-right)
- Pre-trained on massive text
- **Key Feature:** Text generation
- **Versions:** GPT-1, GPT-2, GPT-3, GPT-4

</> GPT Example

```
1 from transformers import GPT2Tokenizer,
   GPT2LMHeadModel
2
3 tokenizer = GPT2Tokenizer.from_pretrained(
4     'gpt2'
5 )
6 model = GPT2LMHeadModel.from_pretrained(
7     'gpt2'
8 )
9
10 prompt = "Voter turnout in elections"
11 inputs = tokenizer(
12     prompt,
13     return_tensors='pt'
14 )
```

GPT (Generative Pre-trained Transformer)

⚙️ What is GPT?

- Autoregressive language model
- Unidirectional (left-to-right)
- Pre-trained on massive text
- **Key Feature:** Text generation
- **Versions:** GPT-1, GPT-2, GPT-3, GPT-4

</> GPT Example

```
1 # Generate text
2 outputs = model.generate(
3     inputs.input_ids,
4     max_length=50,
5     temperature=0.7,
6     do_sample=True
7 )
8
9 generated_text = tokenizer.decode(
10     outputs[0],
11     skip_special_tokens=True
12 )
13 print(generated_text)
```

Large Language Models (LLMs)

State-of-the-Art LLMs

- GPT-4 (OpenAI)
- Claude (Anthropic)
- Gemini (Google)
- Llama (Meta)
- Mistral
- Falcon

Key Capabilities

- Text generation
- Translation
- Summarization
- Question answering
- Code generation
- Reasoning

LLM Techniques

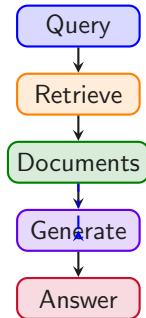
- Prompt Engineering
- Fine-tuning
- RAG
- Few-shot learning
- Zero-shot learning

RAG

Retrieval Augmented Generation (RAG)

What is RAG?

- Combines retrieval and generation
- Retrieve relevant documents
- Generate responses with context
- **Benefits:**
 - Up-to-date information
 - Source attribution
 - Reduced hallucinations



RAG - Code Example

</> RAG Implementation

```
1 from langchain.embeddings import OpenAIEmbeddings
2 from langchain.vectorstores import FAISS
3 from langchain.chains import RetrievalQA
4 from langchain.llms import OpenAI
5
6 # 1. Create embeddings
7 embeddings = OpenAIEmbeddings()
8
9 # 2. Load documents
10 documents = load_documents("election_data/")
11
12 # 3. Create vector store
13 vectorstore = FAISS.from_documents(
14     documents, embeddings
15 )
16
17 # 4. Create retriever
18 retriever = vectorstore.as_retriever(
19     search_kwargs={"k": 3}
20 )
```

RAG - Code Example

</> RAG Implementation

```
1 # 5. Create RAG chain
2 qa_chain = RetrievalQA.from_chain_type(
3     llm=OpenAI(),
4     retriever=retriever,
5     return_source_documents=True
6 )
7
8 # 6. Query
9 query = "What was the voter turnout in Lagos in 2023?"
10 result = qa_chain(query)
11
12 print(f"Answer: {result['result']}")
13 print(f"Sources: {result['source_documents']}")
```

NLP for INEC

NLP Applications at INEC

Chatbots & FAQ

- Voter education chatbot
- Multilingual support
- 24/7 query handling
- Automated responses

Misinformation Detection

- Detect fake news
- Social media monitoring
- Fact-checking support
- Disinformation triage

Document Analysis

- Extract key information
- Summarize reports
- Sentiment analysis
- Named entity extraction

Misinformation Detection with NLP

Detection Approaches

- **Content Analysis** – Linguistic patterns
- **Sentiment Analysis** – Emotional language
- **Named Entity Recognition** – Verify entities
- **Source Analysis** – Check reliability

</> Code Example

```
1 from transformers import pipeline
2
3 # Load pre-trained model
4 classifier = pipeline(
5     "text-classification",
6     model="roberta-base-openai-detector"
7 )
8
9 # Check text for misinformation
10 text = "Election results have been manipulated!"
11 result = classifier(text)
12
13 print(f"Prediction: {result[0]['label']}")
14 print(f"Confidence: {result[0]['score']:.2f}")
15
16 # Alternative: Use domain-specific model
17 # model = AutoModelForSequenceClassification.
18 #     from_pretrained(
19 #         "misinformation-detection-model"
```

Key Takeaways

Key Takeaways

What We Covered

1. **NLP Fundamentals** – Understanding language
2. **Preprocessing** – Text cleaning techniques
3. **Traditional NLP** – BoW, TF-IDF, Word Embeddings
4. **Transformers** – Attention is all you need
5. **BERT, GPT, LLMs** – Modern NLP models
6. **RAG** – Retrieval augmented generation
7. **INEC Applications** – Real-world use cases

💡 Key Principles

- ✓ **Context Matters** – Understanding semantics
- ✓ **Data Quality** – Clean text data
- ✓ **Model Selection** – Choose appropriate model
- ✓ **Fine-tuning** – Adapt to domain
- ✓ **Evaluation** – Measure performance
- ✓ **Ethics** – Bias and fairness

Resources for Further Learning

Online Courses

-  Coursera: NLP Specialization (DeepLearning.AI)
-  Hugging Face NLP Course
-  FastAI: Practical Deep Learning

Tools & Libraries

-  Hugging Face Transformers
-  LangChain
-  NLTK & spaCy
-  PyTorch / TensorFlow

Books

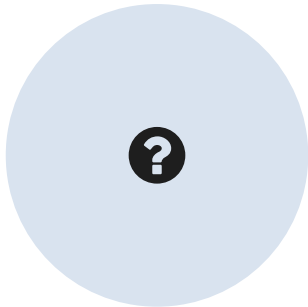
-  *Speech and Language Processing* – Jurafsky & Martin
-  *Natural Language Processing with Transformers*
-  *Deep Learning for NLP*

Communities

-  Hugging Face Forums
-  ACL NLP Community

Thank You

Questions & Discussion



"Language is the foundation of human intelligence."